

```

import os
import numpy as np
import pandas as pd
import cv2
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from sklearn.model_selection import train_test_split, StratifiedShuffleSplit
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

# =====
# STEP 1: Load Dataset & Remove Corrupt Images
# =====

IMAGE_FOLDER = r'C:\Users\Gurbachan\Desktop\CV_ASS2\image\photos' # Update this
LABELS_FILE = r'C:\Users\Gurbachan\Desktop\CV_ASS2\image\photos.csv' # Update this

df = pd.read_csv(LABELS_FILE)

# Ensure required columns exist
if 'photo_id' not in df.columns or 'label' not in df.columns:
    raise ValueError("✗ CSV file must contain 'photo_id' and 'label' columns!")

df['photo_id'] = df['photo_id'].astype(str) # Ensure photo_id is string

# Fix missing extensions
image_files = set(os.listdir(IMAGE_FOLDER))
for ext in [".jpg", ".png", ".jpeg"]:
    df["photo_id_ext"] = df["photo_id"] + ext
    df["exists"] = df["photo_id_ext"].isin(image_files)
    if df["exists"].sum() > 0:
        break

df_valid = df[df["exists"]].copy()

# Encode Labels
label_encoder = LabelEncoder()
df_valid['label_encoded'] = label_encoder.fit_transform(df_valid['label'])

print(f"✅ After removing corrupt images: {df_valid.shape[0]} valid images remaining!")

# =====
# STEP 2: Stratified Sampling
# =====

SAMPLE_SIZE = 20000

sss = StratifiedShuffleSplit(n_splits=1, test_size=0.8, random_state=42)
for train_index, _ in sss.split(df_valid, df_valid['label_encoded']):
    df_sampled = df_valid.iloc[train_index]

print("\n ♦ After Stratified Sampling Class Distribution:")
print(df_sampled['label'].value_counts())

# =====
# STEP 3: Convert Images to Feature Vectors
# =====

IMG_SIZE = 64
X, y = [], []

for _, row in tqdm(df_sampled.iterrows(), total=len(df_sampled), desc="Processing Images"):
    img_path = os.path.join(IMAGE_FOLDER, row["photo_id_ext"])
    label = row['label_encoded']

    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        continue

```



```

# Predict on Test Data
y_pred_dnn_probs = model_dnn.predict(X_test)
y_pred_dnn = np.argmax(y_pred_dnn_probs, axis=1)

# =====
# 🚀 Step 1: Compute Confusion Matrix
# =====
conf_mat_dnn = confusion_matrix(y_test, y_pred_dnn)
plt.figure(figsize=(8,6))
sns.heatmap(conf_mat_dnn, annot=True, fmt="d", cmap="Blues", xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("DNN Confusion Matrix")
plt.show()

# =====
# 🚀 Step 2: Compute Classification Report with Metrics Display
# =====
report_dnn = classification_report(y_test, y_pred_dnn, target_names=label_encoder.classes_, output_dict=True)

precision_dnn = [report_dnn[label]["precision"] for label in label_encoder.classes_]
recall_dnn = [report_dnn[label]["recall"] for label in label_encoder.classes_]
f1_score_dnn = [report_dnn[label]["f1-score"] for label in label_encoder.classes_]

# Print Classification Report
print("\n 🚀 DNN Classification Report:")
print(classification_report(y_test, y_pred_dnn, target_names=label_encoder.classes_))

# =====
# 🚀 Step 3: Plot AUC-ROC Curve with Values
# =====
plt.figure(figsize=(10, 8))

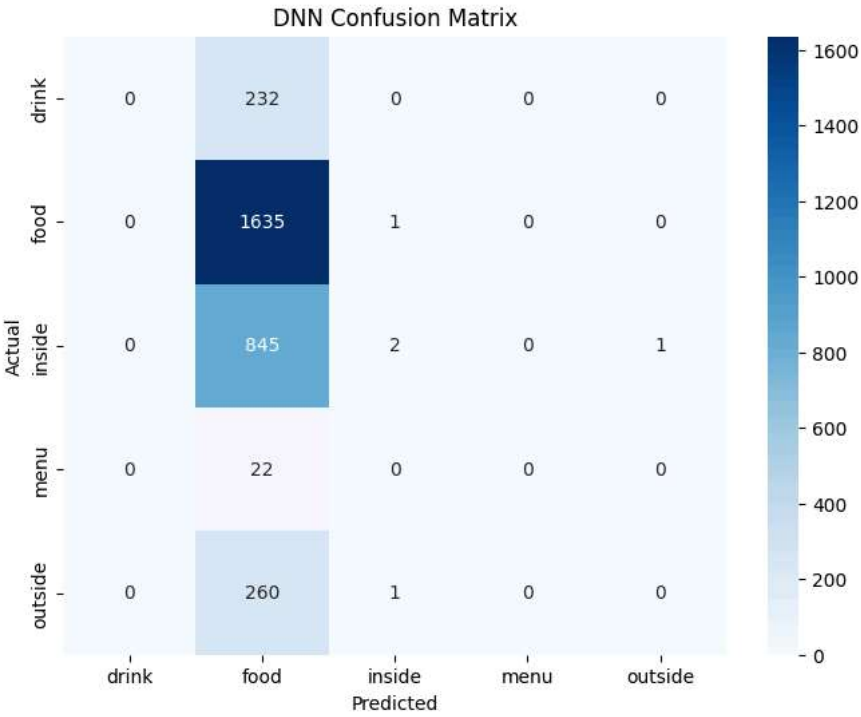
for i in range(len(label_encoder.classes_)):
    fpr, tpr, _ = roc_curve(y_test == i, y_pred_dnn_probs[:, i])
    roc_auc = auc(fpr, tpr)

    plt.plot(fpr, tpr, label=f"{label_encoder.classes_[i]} (AUC={roc_auc:.2f}, P={precision_dnn[i]:.2f}, R={recall_dnn[i]:.2f}, F1={f1_score_dnn[i]:.2f})")

plt.plot([0, 1], [0, 1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("DNN AUC-ROC Curve")
plt.legend(loc="lower right")
plt.show()

```

↺ 94/94 1s 8ms/step



```
C:\Users\Gurbachan\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gurbachan\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gurbachan\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gurbachan\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gurbachan\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\Gurbachan\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-def
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

◆ DNN Classification Report:

	precision	recall	f1-score	support
drink	0.00	0.00	0.00	232
food	0.55	1.00	0.71	1636
inside	0.50	0.00	0.00	848
menu	0.00	0.00	0.00	22
outside	0.00	0.00	0.00	261
accuracy			0.55	2999
macro avg	0.21	0.20	0.14	2999
weighted avg	0.44	0.55	0.39	2999

